UNIVERSITY OF RWANDA

College Of Science and Technology

Computer Engineering

Year *4*

Gako Campus

Reg n°: **223026685**

223026674

2230267476

Names: Jerome HAKIZIMANA

Lewis NGENDAHIMANA

Clement NZABAMWITA

Contents

Note on method and measurement 3
Mini Project 3.1: Simulating RTOS Concurrency, Synchronization and Race Conditions 3
1. Objective 3
2. Procedure and scripts 3
3. Results 4
3.1 Race condition and its repair (three runs each) 4
3.2 Bounded queue (message-queue analogy) 5
3.3 Counting semaphore pool 5
3.4 Dispatch-latency measurement 6
4. Lab task answers 6
Mini Project 3.2: Bounded-Buffer Producer-Consumer with Deadlock Reproduction and Lock-Ordering Fix 10
1. Design 10
2. Reproduced deadlock (flawed lock order) 10
3. Corrected version (global lock ordering) 11
4. Throughput versus buffer capacity (corrected version, mean of 3 runs each) 11
5. Analysis 12
5.1 Mapping Coffman's four conditions to the code 12
5.2 Throughput results 13
5.3 Limitations 13
Mini Project 3.3: Priority-Based Interrupt Latency Simulator 14
1. Design 14
2. Results 14
2.1 Response latency by configuration 14
2.2 Arrival-rate sweep: effect of the highest-priority source's period on the lowest-priority source (ETH) 15
3. Analysis 16
3.1 Baseline behaviour 16
3.2 Why a long top half degrades the other sources 17
3.3 Arrival-rate sweep and schedulability 17
3.4 Limitations 18
Conclusion 19
Appendix A: Source Code 20

CE80763: Real-Time Systems

Unit 3 Lab Report

Note on method and measurement

Python threads are used as stand-ins for RTOS tasks. CPython has no priority-based preemptive scheduler and the Global Interpreter Lock serializes bytecode, so absolute timings here are not RTOS timings; they are used to reproduce the logical behaviours of Unit 3 (lost updates, mutual exclusion, blocking, deadlock, priority preemption, latency). Timings come from time.perf_counter and depend on the host operating system, so exact numbers will differ on another machine; the trends are what matter. Where a measured value reveals a limitation of the simulation, it is stated explicitly.

Mini Project 3.1: Simulating RTOS Concurrency, Synchronization and Race Conditions

1. Objective

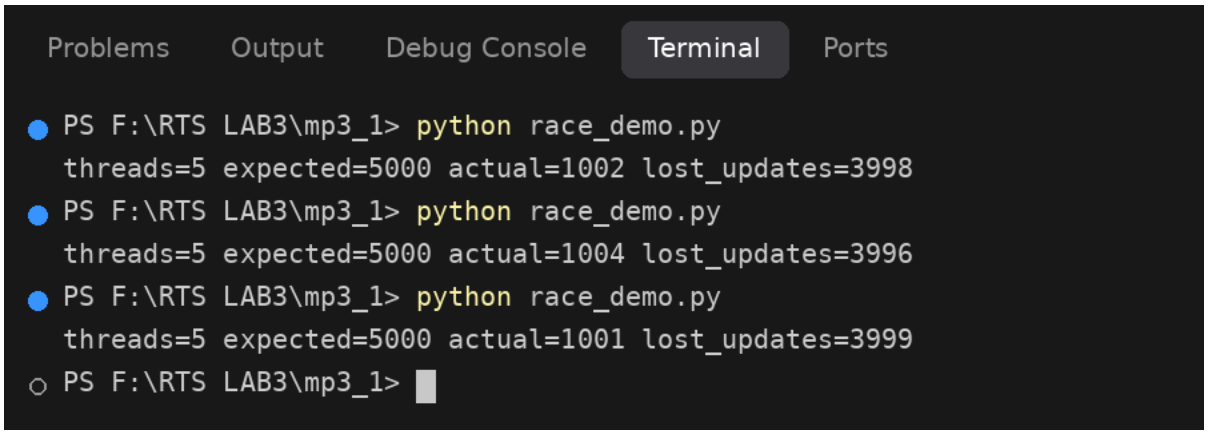
To reproduce, diagnose and repair a race condition (Section 3.8), model a counting semaphore resource pool and bounded message queue (Sections 3.4 and 3.6), and measure dispatch latency as an analogy to context-switch and interrupt response latency (Sections 3.2 and 3.5).

2. Procedure and scripts

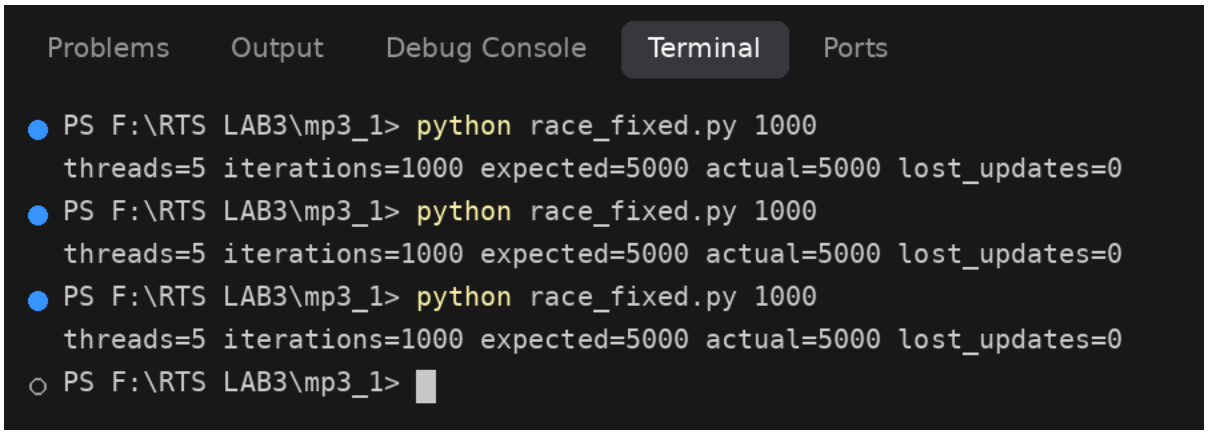
Script	Purpose	Unit 3 concept
race_demo.py	5 threads x 1000 unsynchronized read-sleep-write increments	Race condition (3.8)
race_fixed.py	Same work inside "with counter_lock"	Mutex mutual exclusion (3.4)
queue_sim.py	Bounded queue.Queue(maxsize=5), 20 timestamped items, latency stats	Message queue, response latency (3.4, 3.5)
semaphore_pool.py	Semaphore(3), 8 task threads, per-task wait time	Counting semaphore, memory pool (3.4, 3.6)
timing_overhead.py	200 thread start-to-Event dispatch timings	Context switch / ISR latency analogy (3.2, 3.5)
livelock_demo.py	Lab task: opposite-order locks with lockstep back-off	Livelock (3.8)

3. Results

3.1 Race condition and its repair (three runs each)



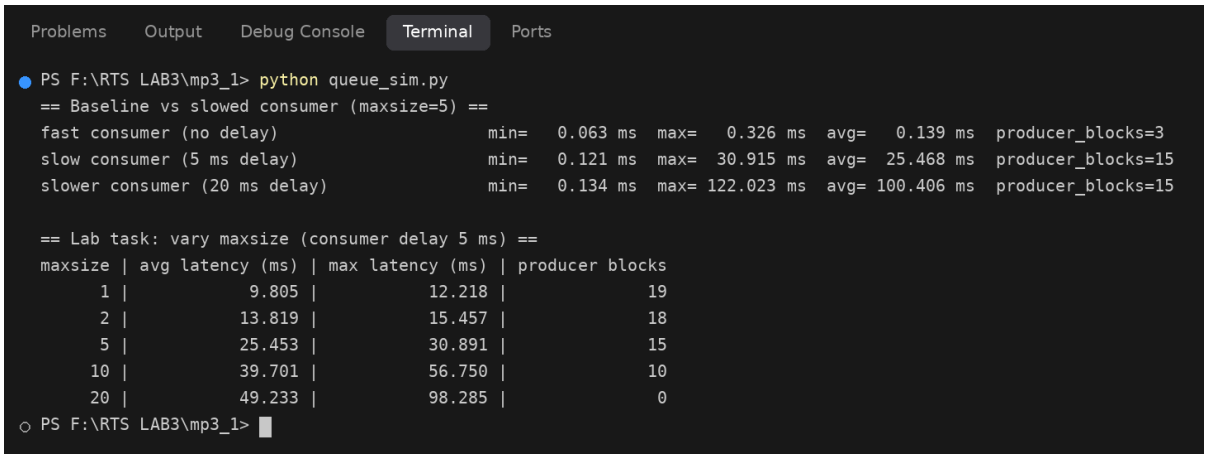
Screenshot 1: race_demo.py, three runs: the unsynchronized counter loses most updates and varies.



Screenshot 2: race_fixed.py, three runs: the lock makes the result exactly 5000 every time.

Every unsynchronized run lost almost all updates (about 1000 of an expected 5000), and the value varied from run to run (1001, 1002, 1004), whereas the locked version returned exactly 5000 in all three runs. This is the lost-update race of Section 3.8: each task reads the counter, is descheduled during the 0.1 ms sleep, and writes back a stale value that overwrites other tasks' increments. The final value is close to 1000 rather than a random fraction of 5000 because the sleep makes all five threads read the same value at nearly the same instant in every round; effectively one increment survives per round of 1000 rounds. The deliberate sleep exaggerates the effect compared with a real few-instruction window, but a race window of any width produces the same class of bug, only less frequently and therefore harder to reproduce.

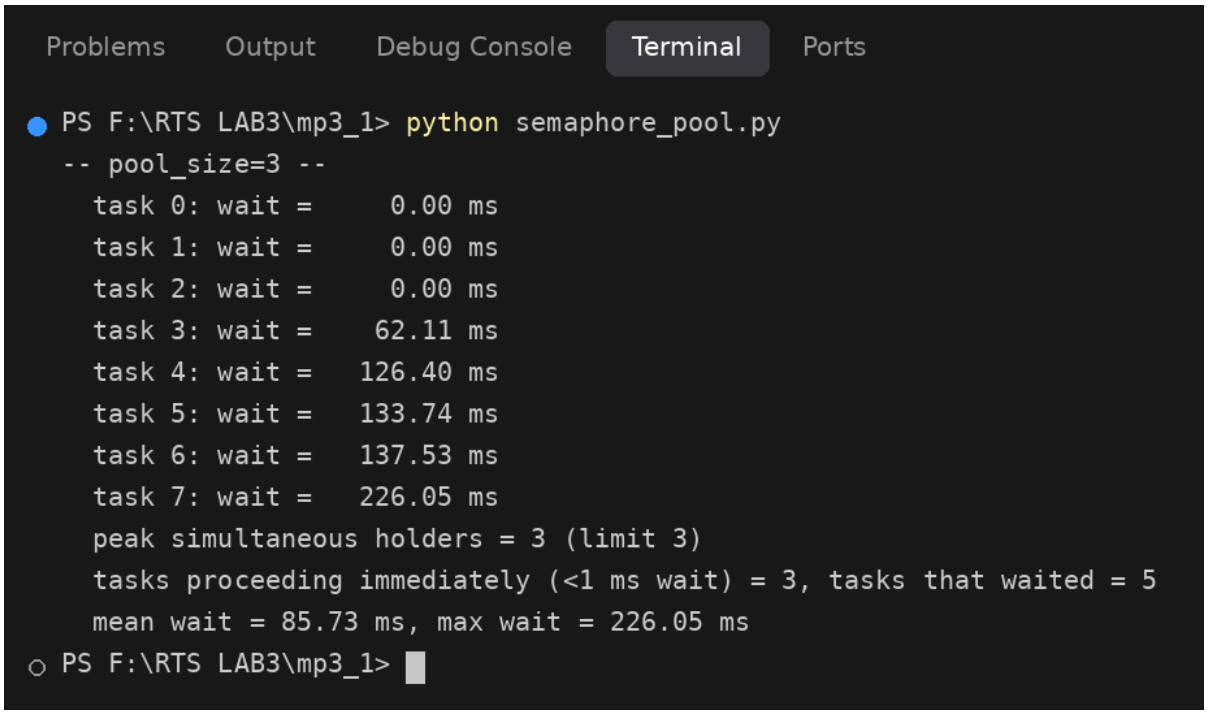
3.2 Bounded queue (message-queue analogy)



Screenshot 3: queue_sim.py: latency and producer blocking with fast and slowed consumers, plus the maxsize sweep (Lab Task 4).

With an unthrottled consumer the average send-to-receive latency was 0.139 ms. Adding a 5 ms consumer delay raised the average latency to 25.5 ms and the maximum to 30.9 ms, and producer blocking events rose from 3 to 15; a 20 ms delay raised the average to 100.4 ms. Once the consumer is slower than the producer, the queue sits full, every new item waits behind up to five others, and the producer is repeatedly forced to block. This is the response-latency effect of Section 3.5: delay is dominated by how long deferred work takes to be serviced, not by how fast the data was handed over.

3.3 Counting semaphore pool

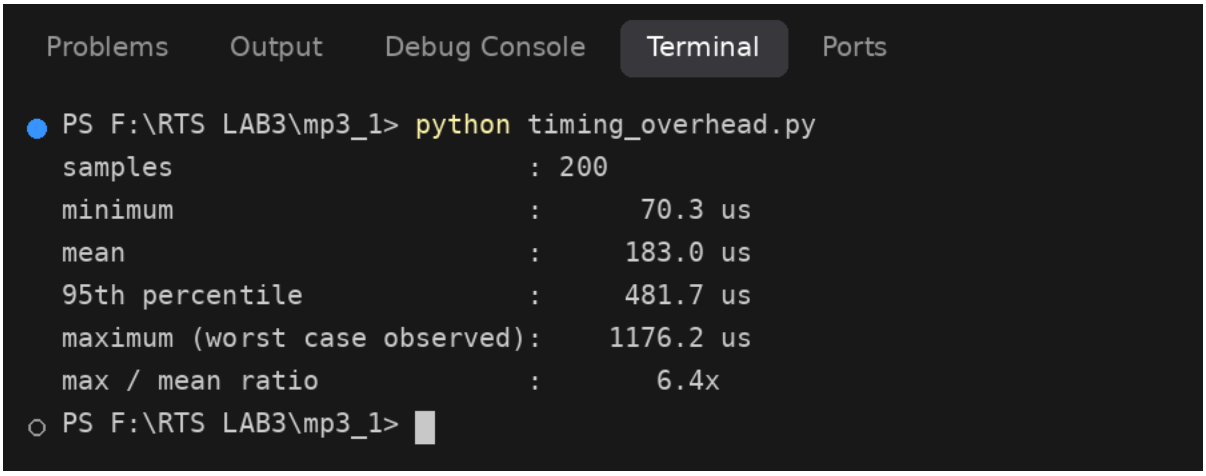


Screenshot 4: semaphore_pool.py with Semaphore(3): three tasks run immediately, five wait, peak holders = 3.

Exactly three tasks proceeded with zero wait; the other five waited between 62 and 226 ms. The peak number of simultaneous holders was 3, never more, confirming that the counting semaphore enforces the pool limit (as a fixed-block memory pool does, Section 3.6).

3.4 Dispatch-latency measurement

Statistic (200 samples)	Value (microseconds)
Minimum	70.3
Mean	183.0
95th percentile	481.7
Maximum (worst case observed)	1176.2
Maximum / mean	6.4x



Screenshot 5: timing_overhead.py: dispatch-latency statistics over 200 samples.

The mean dispatch latency is small, but the worst case is 6.4 times the mean and the 95th percentile is 2.6 times the mean. This is exactly why Section 3.1 argues that real-time engineers must reason about the worst case: a system sized on the 183 microsecond average would be wrong by a factor of more than six on the unluckiest sample, and a longer test would probably find an even larger maximum.

4. Lab task answers

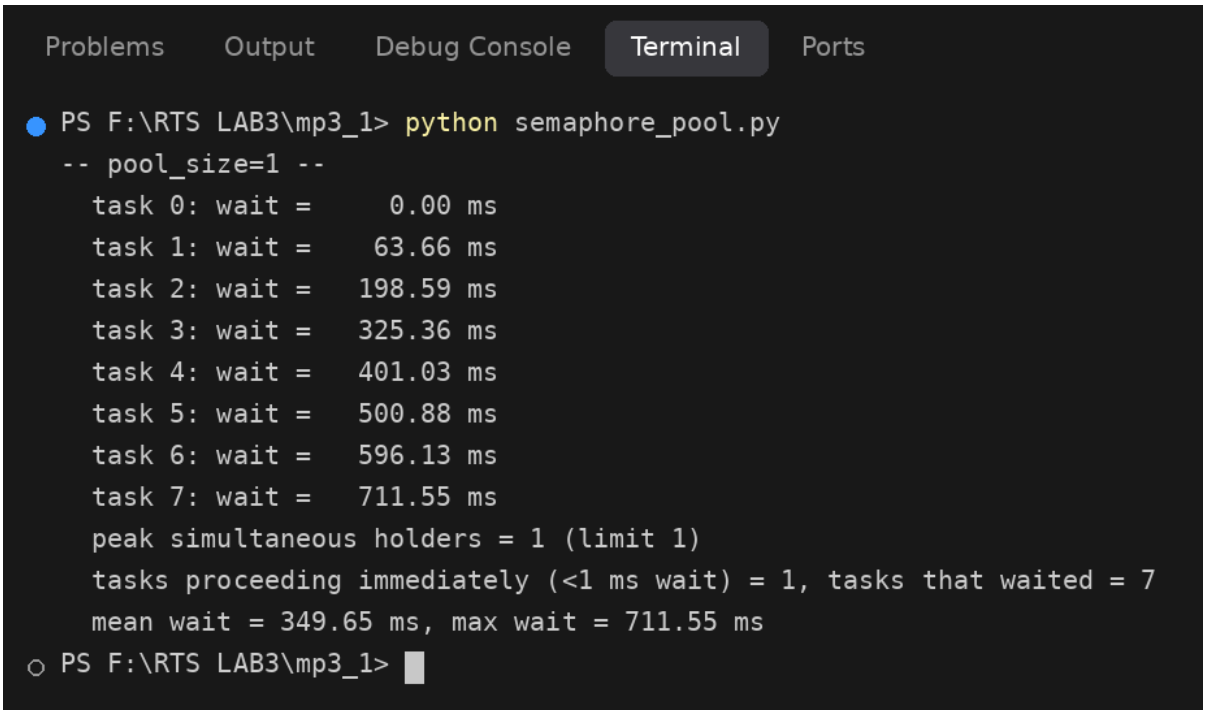
Task 1: ten threads instead of five. Ten threads gave 1009 against an expected 10000 (see below), i.e. 8991 lost updates (90%) compared with 3998 of 5000 (80%) for five threads. The loss grows in absolute terms and as a fraction: more tasks interleave over the same unprotected read-modify-write window, so more writes land on stale values. Adding concurrency to an unprotected shared variable makes the bug worse, not better.



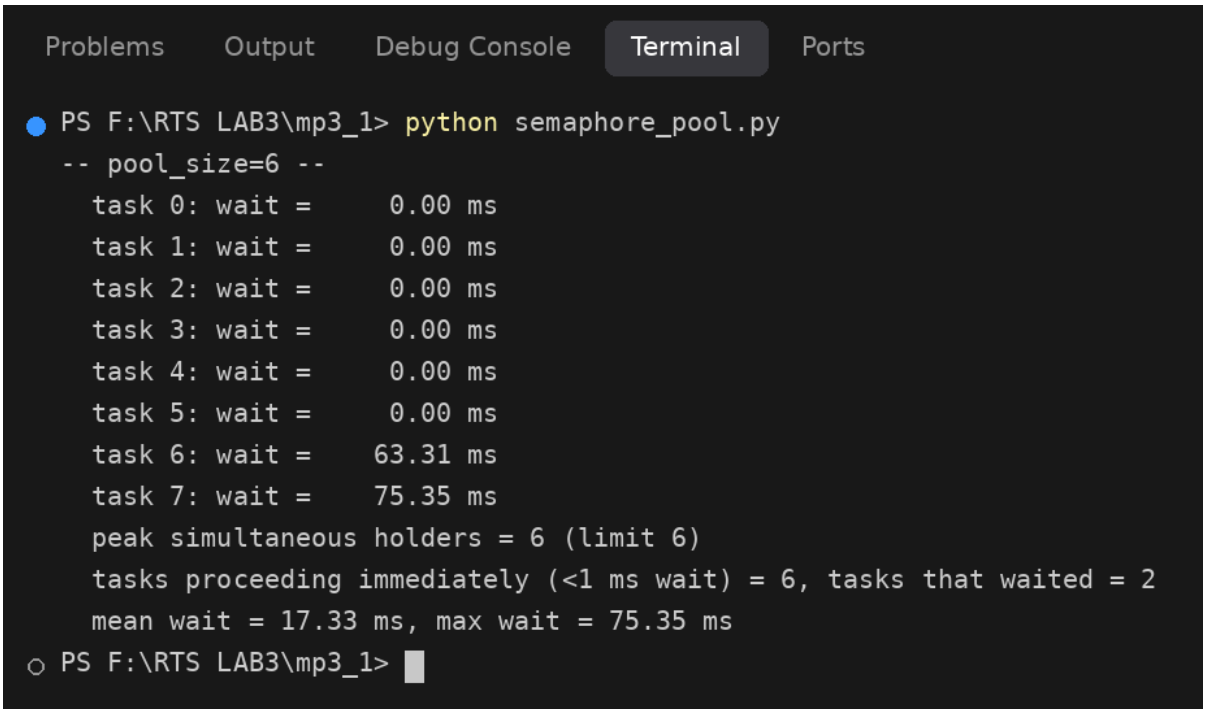
Screenshot 6: race_demo.py with 10 threads (Lab Task 1).

Task 2: semaphore count 1 and 6. Results for the three pool sizes (8 tasks, identical seeded hold times):

Pool size	Tasks with no wait	Tasks that waited	Mean wait (ms)	Max wait (ms)	Peak holders
1	1	7	349.65	711.55	1
3	3	5	85.73	226.05	3
6	6	2	17.33	75.35	6



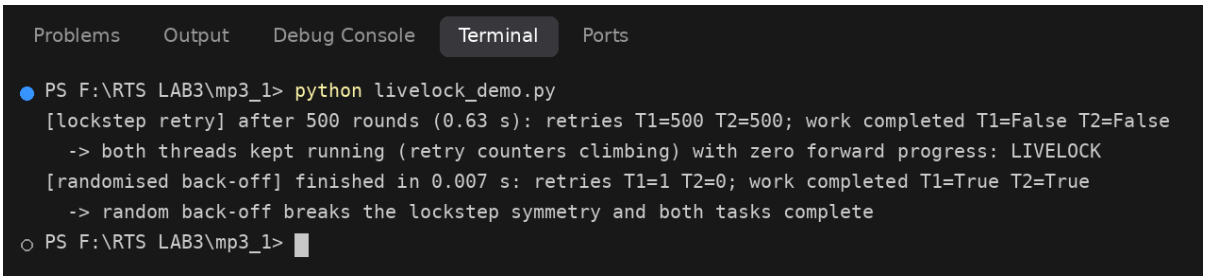
Screenshot 7: semaphore_pool.py with Semaphore(1) (Lab Task 2): fully serialized.



Screenshot 8: semaphore_pool.py with Semaphore(6) (Lab Task 2): only two tasks wait.

With a count of 1 the semaphore degenerates into a binary semaphore/mutex-like lock: tasks are fully serialized, waits accumulate (task 7 waited 712 ms, the sum of seven earlier holds) and contention is maximal. With a count of 6 only two tasks must wait. Wait time falls roughly in proportion to the number of identical resources, which is the counting-semaphore behaviour of Section 3.4: it tracks availability of N interchangeable units and blocks callers only when the count reaches zero.

Task 3: livelock. Two threads take two locks in opposite orders, release the first lock politely when the second is unavailable, and retry in lockstep (barrier-synchronized). Output:



Screenshot 9: livelock_demo.py (Lab Task 3): both threads retry 500 times with no progress; random back-off resolves it.

Each thread performed 500 retries and neither ever completed its work. Both stayed actively executing (their retry counters kept rising), so this is livelock, not deadlock: no thread was blocked, yet no forward progress was made, matching the definition in Section 3.8. Adding a randomized back-off delay broke the lockstep symmetry and both tasks finished after one or zero retries in about 7 ms, the mitigation recommended in Section 3.8.

Task 4: queue maxsize sweep (consumer delay 5 ms, 20 items).

maxsize	Avg latency (ms)	Max latency (ms)	Producer blocks
1	9.805	12.218	19
2	13.819	15.457	18
5	25.453	30.891	15
10	39.701	56.750	10
20	49.233	98.285	0

A larger queue reduces producer blocking (19 blocks at size 1, none at size 20 because all 20 items fit) but increases per-item latency (9.8 ms to 49.2 ms), because items queue up behind a slow consumer. Buffer size therefore trades producer decoupling against end-to-end latency; for a deadline-driven RTOS design a small bounded queue keeps worst-case latency low and makes overload visible early instead of hiding it in a long backlog.

Task 5: why a plain semaphore is unsuitable where priority-inheritance-style reasoning is required. Section 3.4 states that a semaphore is ownerless: the task that gives it need not be the one that took it. A mutex has an explicit owner, which lets the kernel know which task holds the resource and temporarily raise that task's priority to that of a higher-priority task blocked on it (priority inheritance, Unit 2 and Section 3.10). If a semaphore guards a shared resource the kernel has no owner to boost, so a medium-priority task can preempt the low-priority holder indefinitely and cause unbounded priority inversion, which is the failure traced in Section 3.14. A caveat on the simulation: Python's threading.Lock is itself not owner-checked (another thread may release it) and Python has no thread priorities, so it only models the mutual-exclusion semantics; threading.RLock tracks an owner, which is the closer analogue of an RTOS mutex.

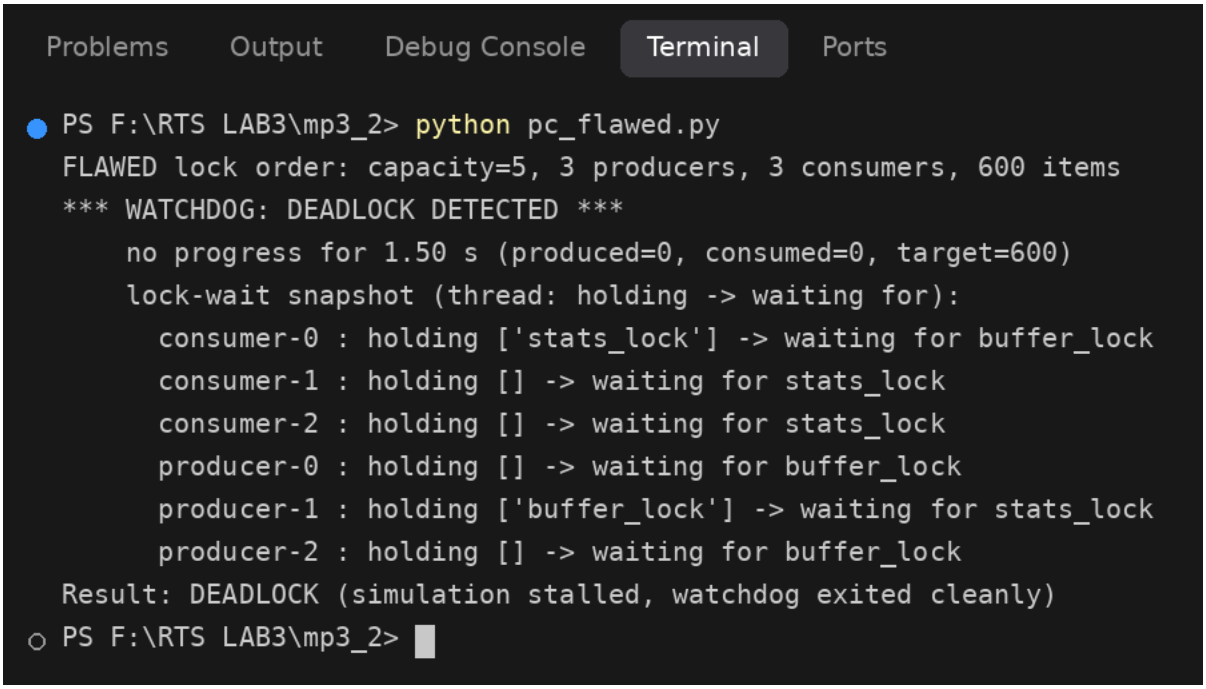
Mini Project 3.2: Bounded-Buffer Producer-Consumer with Deadlock Reproduction and Lock-Ordering Fix

1. Design

The simulation (pc_core.py, with entry points pc_flawed.py, pc_fixed.py and pc_sweep.py) has three producer and three consumer threads sharing a bounded buffer (a Python list with a configurable capacity, default 5) guarded by buffer_lock, and a statistics dictionary (items produced, items consumed, total wait time) guarded by stats_lock. Both locks are explicit threading.Lock objects, so acquisition order is fully controlled. Each task holds its first lock for 0.5 ms before requesting its second, which widens the hold-and-wait window. A watchdog thread samples the produced and consumed counters every 0.1 s and, if neither has changed for 1.5 s before completion, prints a diagnosis including a lock-wait snapshot and ends the run without any manual kill. Capacity, producer and consumer counts, and total items are command-line or Config parameters.

2. Reproduced deadlock (flawed lock order)

In the flawed version producers acquire buffer_lock then stats_lock, while consumers acquire stats_lock then buffer_lock. Captured output:



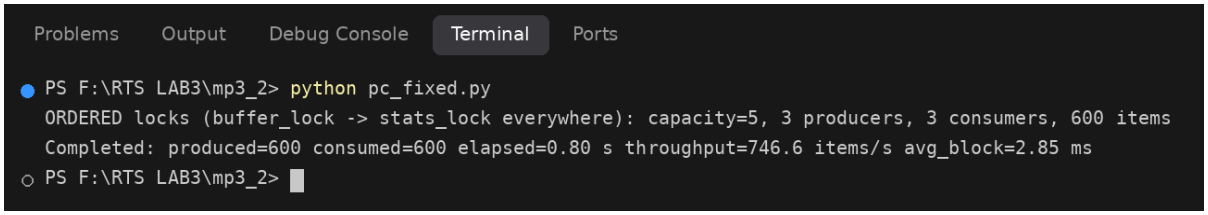
Screenshot 10: pc_flawed.py: watchdog reports the deadlock and prints the lock-wait snapshot.

The snapshot is a textbook circular wait. producer-1 holds buffer_lock and waits for stats_lock, while consumer-0 holds stats_lock and waits for buffer_lock. The remaining four threads are queued behind these two. No thread can proceed and nothing can be taken away from the holders, so progress stopped. An honest note: with these timings the deadlock formed at the very start of the run (produced = 0, consumed = 0) rather than after sustained load, because all six tasks begin simultaneously and the 0.5 ms hold gap makes the opposing acquisition orders

overlap almost immediately. The behaviour is reproducible on every run we tried; in a real system with shorter hold windows the same flaw would appear intermittently under load, which makes it harder to find than here.

3. Corrected version (global lock ordering)

In the corrected version every task acquires buffer_lock before stats_lock. The identical workload then completes:

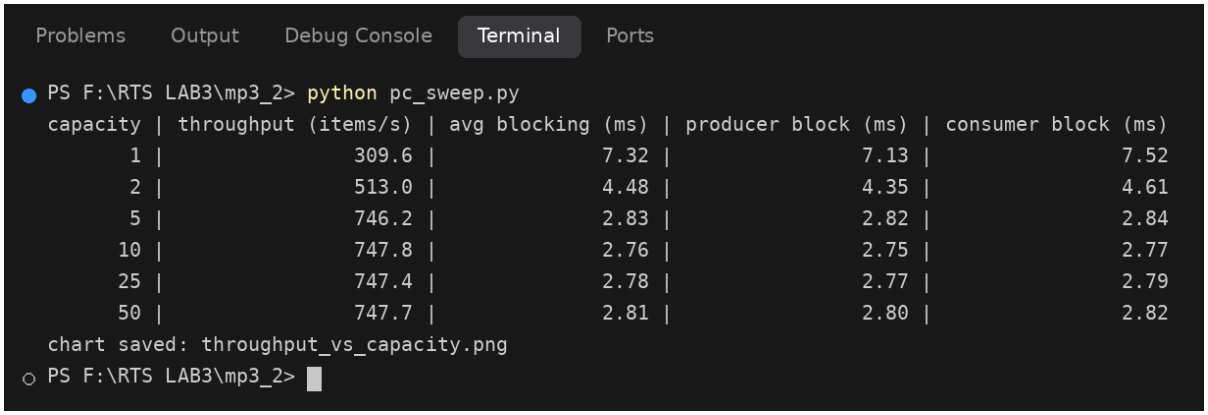


Screenshot 11: pc_fixed.py: with a global lock order the same workload completes.

All 600 items were produced and consumed, the watchdog never fired, and no timed-out or aborted threads were needed.

4. Throughput versus buffer capacity (corrected version, mean of 3 runs each)

Capacity	Throughput (items/s)	Avg blocking (ms)	Producer block (ms)	Consumer block (ms)
1	309.6	7.32	7.13	7.52
2	513.0	4.48	4.35	4.61
5	746.2	2.83	2.82	2.84
10	747.8	2.76	2.75	2.77
25	747.4	2.78	2.77	2.79
50	747.7	2.81	2.80	2.82



Screenshot 12: pc_sweep.py: throughput and blocking time for six buffer capacities.

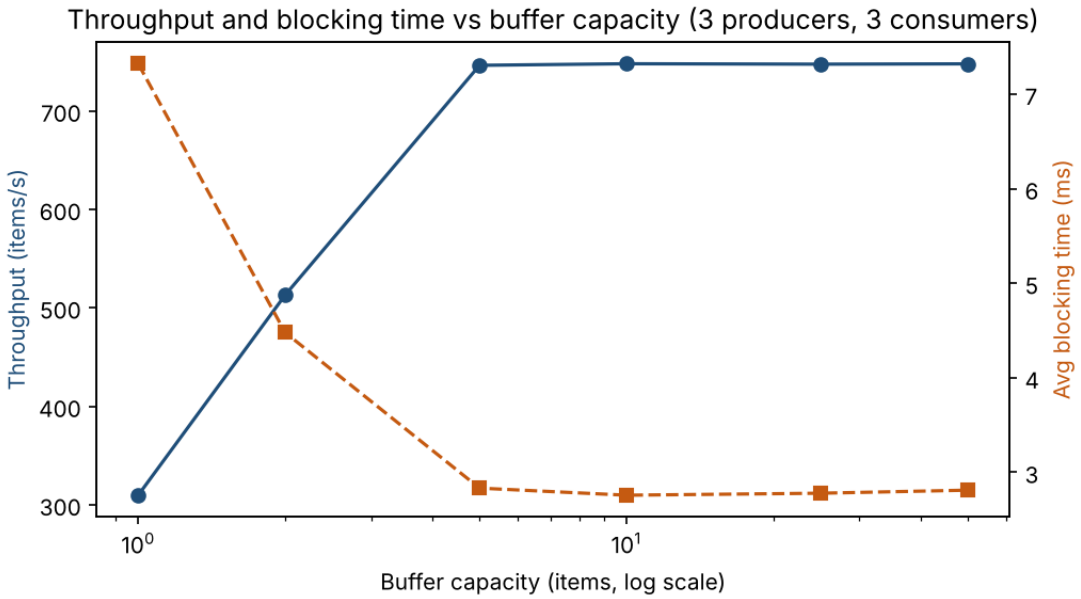


Figure 1: Throughput and average blocking time versus buffer capacity.

5. Analysis

5.1 Mapping Coffman's four conditions to the code

Condition (Section 3.8)	Where it is satisfied in the flawed code
Mutual exclusion	buffer_lock and stats_lock are threading.Lock objects; only one thread may hold each at a time.
Hold-and-wait	In producer() and consumer() the task calls acquire() on its first lock, sleeps hold_gap, then calls acquire() on its second lock while still holding the first.
No preemption	A Python lock cannot be taken from its holder; it is released only by the holder's own release() call.
Circular wait	Producers request buffer_lock then stats_lock; consumers request stats_lock then buffer_lock. A producer holding buffer_lock and a consumer holding stats_lock wait on each other (visible in the watchdog snapshot).

All four conditions hold in the flawed version, so deadlock is possible, and the experiment shows it is actually reached. The fix removes only the fourth condition. Once every task takes buffer_lock first and stats_lock second, a task can wait for stats_lock only while holding buffer_lock, and a task holding stats_lock never waits for buffer_lock; the "waits-for" relation therefore cannot form a cycle regardless of timing. Mutual exclusion, hold-and-wait and no-preemption are all still present, which is deliberate: removing mutual exclusion would reintroduce the race conditions of Section 3.8, and removing hold-and-wait or adding preemption would need more machinery (all-or-nothing acquisition, or forced lock reclamation) for no extra benefit here. Global lock ordering is the most commonly targeted elimination in practice because it costs nothing at run time and only needs a documented convention that can be code-reviewed.

5.2 Throughput results

With a capacity of 1, producers and consumers must strictly alternate: a producer cannot insert until a consumer has removed the previous item, and vice versa. Throughput was only 309.6 items/s and the average blocking time was 7.32 ms per item. Doubling the capacity to 2 raised throughput by 66% (to 513.0 items/s) and cut blocking by 39%. Moving to capacity 5 gave a further 45% gain (746.2 items/s) and brought blocking to 2.83 ms. Beyond 5, throughput stayed flat at about 747 items/s (747.8, 747.4 and 747.7 for capacities 10, 25 and 50) and blocking time stayed at about 2.8 ms. The point of diminishing returns is therefore a capacity of about 5, which is also the default; larger buffers cost memory and add no throughput.

The plateau tells us that the buffer is no longer the bottleneck once it can absorb the short-term rate mismatch between six tasks. What limits throughput then is the serialized critical section: every insert or removal holds buffer_lock (and stats_lock) for at least the 0.5 ms hold gap plus scheduling overhead, so the six threads share a single lock that can only complete roughly 1,500 to 1,800 operations per second in total, and each item needs two operations (insert and remove). That is consistent with the measured ceiling of about 750 items/s, although we did not instrument lock-hold time separately to prove it. The remaining blocking time (about 2.8 ms) is mostly queueing for the lock, not waiting for buffer space, which is why producer and consumer blocking are nearly identical at large capacities. The practical RTOS lesson matches Section 3.4 and Section 3.10: size a bounded buffer to absorb bursts and then shorten critical sections, because past that point only shorter critical sections improve throughput.

5.3 Limitations

Sleep-based work times and the 0.5 ms hold gap make absolute numbers specific to this host and Python scheduling (sleep granularity is about 1 ms on many systems). The producer and consumer retry loops back off for 0.5 ms when the buffer is full or empty; this polling is a simplification of a blocking queue and contributes to the measured blocking time at small capacities. Results are the mean of three runs per capacity and carried no confidence intervals.

Mini Project 3.3: Priority-Based Interrupt Latency Simulator

1. Design

irq_sim.py models three periodic interrupt sources, each with a top half (ISR) and a bottom-half task, on one simulated CPU. All times are in milliseconds from time.perf_counter.

Source	Priority (1 = highest)	Period (ms)	Top half (ms)	Bottom half (ms)
UART	1	20 (swept 40 to 8)	0.5	3.0
ADC	2	50	0.5	8.0
ETH	3	100	1.0	15.0

- Each source is a thread that sleeps until its next release time, records the assertion timestamp, and runs its top half. Interrupt latency is the time from assertion to the start of the top half.
- Top halves always preempt bottom-half work: the CPU worker runs bottom halves in 1 ms slices (the modelled interrupt-masked window) and yields to any pending top half between slices.
- At the end of a top half a bottom-half record (priority, assertion time, remaining duration) is pushed on a heapq priority queue. One CPU worker always pops the highest-priority ready bottom half, runs one slice and re-queues the remainder, so a bottom half can be preempted mid-execution by a newly arrived higher-priority bottom half.
- Response latency is bottom-half completion time minus assertion time, as defined in Section 3.5.
- Simplification: top halves are serialized (a top half waits for the running top half to finish); nested hardware interrupt priorities are not modelled.

2. Results

Each of the three configurations ran for 10 s. Latencies are in milliseconds.

2.1 Response latency by configuration

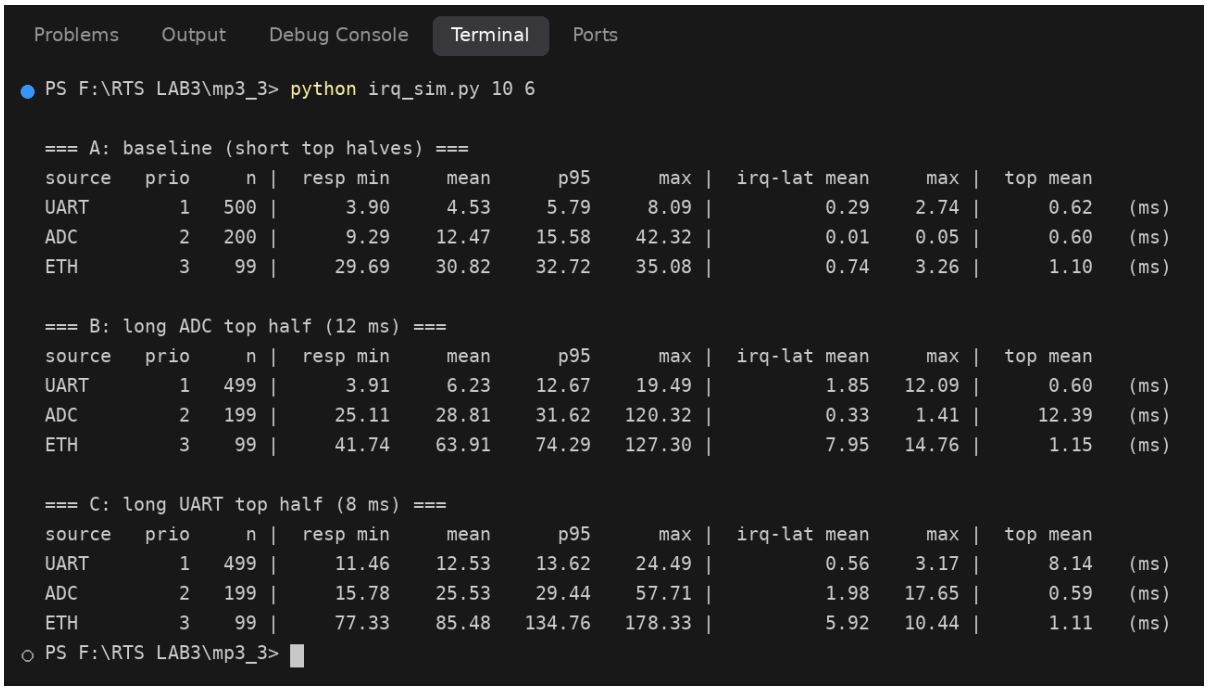
Config	Source	n	Min	Mean	P95	Max	IRQ lat mean	IRQ lat max
A baseline	UART	500	3.90	4.53	5.79	8.09	0.29	2.74
A baseline	ADC	200	9.29	12.47	15.58	42.32	0.01	0.05
A baseline	ETH	99	29.69	30.82	32.72	35.08	0.74	3.26
B long ADC top half (12 ms)	UART	499	3.91	6.23	12.67	19.49	1.85	12.09
B long ADC top half (12 ms)	ADC	199	25.11	28.81	31.62	120.32	0.33	1.41
B long ADC top half (12 ms)	ETH	99	41.74	63.91	74.29	127.30	7.95	14.76
C long UART top half (8 ms)	UART	499	11.46	12.53	13.62	24.49	0.56	3.17

Config	Source	n	Min	Mean	P95	Max	IRQ lat mean	IRQ lat max
C long UART top half (8 ms)	ADC	199	15.78	25.53	29.44	57.71	1.98	17.65
C long UART top half (8 ms)	ETH	99	77.33	85.48	134.76	178.33	5.92	10.44

2.2 Arrival-rate sweep: effect of the highest-priority source's period on the lowest-priority source (ETH)

UART period (ms)	UART load (%)	ETH n	ETH mean (ms)	ETH P95 (ms)	ETH max (ms)
40	8.8	59	25.53	28.30	47.98
20	17.5	59	30.90	32.29	32.70
12	29.2	59	37.32	40.52	66.02
8	43.8	59	71.86	80.12	91.44

UART load is (top half + bottom half) / period = 3.5 ms / period. Each sweep point ran for 6 s.



Screenshot 13: irq_sim.py: response-latency tables for configurations A, B and C.

```
Problems  Output  Debug Console  Terminal  Ports

PS F:\RTS LAB3\mp3_3>
=== Sweep: UART period vs ETH (lowest priority) response latency ===
 UART period (ms)  UART load %  ETH n  ETH mean  ETH p95  ETH max
             40.0         8.8     59     25.53    28.30    47.98
             20.0        17.5     59     30.90    32.29    32.70
             12.0        29.2     59     37.32    40.52    66.02
              8.0        43.8     59     71.86    80.12    91.44

timeline saved: timeline.png
PS F:\RTS LAB3\mp3_3>
```

Screenshot 14: irq_sim.py: arrival-rate sweep and timeline plot written to disk.



Figure 2: A 200 ms window of the baseline run. Hatched bars are top halves (ISRs); light bars are 1 ms bottom-half slices. Bottom halves of ADC and ETH are interrupted by UART top halves, and the UART bottom half runs ahead of the lower-priority work.

3. Analysis

3.1 Baseline behaviour

In the baseline the priorities behave as designed. UART, the highest priority source, has a mean response latency of 4.5 ms against its nominal 3.5 ms of work (0.5 ms top half plus 3.0 ms bottom half); the extra millisecond comes from waiting for a slice boundary and from timer overhead (a sleep-based 0.5 ms top half measured 0.62 ms on average). ADC and ETH show progressively larger response latencies (12.5 ms and 30.8 ms) because each is delayed by every higher-priority bottom half that arrives during its execution. Interrupt latency stayed below 3.3 ms for all sources, and for UART its mean was 0.29 ms, which matches the model: the only delay is waiting at most one 1 ms slice. The timeline in Figure 2 shows the mechanism directly: ADC and ETH bottom halves are chopped into slices and pause whenever a UART top half or bottom half becomes ready. One outlier deserves mention: the baseline ADC maximum of 42.3 ms is far above its 95th percentile of 15.6 ms. We did not trace the exact cause, but the likely

explanation is a burst of coinciding releases combined with host operating-system scheduling jitter in the Python threads; it is a reminder that wall-clock simulations on a general-purpose OS can show jitter that the model itself does not contain.

3.2 Why a long top half degrades the other sources

Configuration B gives the middle-priority ADC source a 12 ms top half (measured mean 12.39 ms) and the damage is not confined to ADC. ETH's mean response latency doubled from 30.8 ms to 63.9 ms, its 95th percentile rose from 32.7 ms to 74.3 ms, and its maximum from 35.1 ms to 127.3 ms. Even UART, the highest-priority source, suffered: its mean rose from 4.5 ms to 6.2 ms, its 95th percentile from 5.8 ms to 12.7 ms, and its worst-case interrupt latency to 12.09 ms, essentially the length of the ADC top half. This is the central point of Section 3.5: a top half executes at interrupt level, above all tasks, so while it runs nothing else can run, whatever its priority. Every other computation inherits the top half's execution time as blocking delay. In our simulation UART can be delayed because top halves are serialized; on hardware with nested interrupt priorities the highest-priority interrupt could preempt the ADC ISR, but the bottom halves of every lower-priority source would still be delayed by the 12 ms of ADC ISR time, and any interrupt of equal or lower priority than ADC would be masked during it. Configuration C puts the long top half (8 ms) on the highest-priority source and is even worse for the lowest-priority task: ETH mean 85.5 ms, 95th percentile 134.8 ms and maximum 178.3 ms. UART itself gets a response latency of 12.5 ms against 4.5 ms in the baseline because of its own long top half. In C the long top half runs 50 times per second (every 20 ms), so it consumes 40% of the CPU at interrupt level before any bottom half runs; in B the long top half runs only every 50 ms (24% of the CPU). Hence the frequency of a long ISR matters as much as its length.

Splitting ISR work into a short top half and a schedulable bottom half works because the bottom half is an ordinary prioritized task: its delay can be analysed with the response-time techniques of Unit 2 and it can be preempted by anything more urgent, whereas ISR time is blocking for everyone and cannot be preempted by tasks. In the baseline the total top-half time is only about 0.5 ms per source per release, so the interrupt-level blocking term stays small and known. The long top half configurations break exactly this assumption.

3.3 Arrival-rate sweep and schedulability

The sweep shortens the UART period from 40 ms to 8 ms, increasing its load from 8.8% to 43.8%, while the lowest-priority ETH source is left untouched. ETH's mean response latency rises from 25.5 ms to 30.9 ms, 37.3 ms and then 71.9 ms, and its 95th percentile from 28.3 ms to 80.1 ms. The growth is not linear: from a period of 40 ms to 12 ms (load 8.8% to 29.2%) mean latency rose by 46%, but at 8 ms (load 43.8%) it nearly doubled. When the high-priority load rises, more UART work interrupts each ETH execution, and ETH is also interfered with by ADC; as total utilisation approaches about 77% (43.8% UART, about 17% ADC and about 16% ETH, including top halves) the lowest-priority task spends increasingly long in the ready queue. This reproduces the response-time analysis result of Unit 2: the response time of a task is its own execution time plus the interference of all higher-priority tasks released during its response interval, and that interference grows super-linearly as the interval grows. The first-order behaviour that high-priority work, especially if its execution time or arrival rate is not bounded, determines the worst case of everything beneath it is the principle that the project set out to

demonstrate. If either the length of a top half or the arrival rate of a high-priority interrupt is unbounded (for example a noisy sensor line), then the schedulability guarantees of every lower-priority task are void, which is why Section 3.5 requires ISRs to be short and their worst-case execution time and maximum arrival rate to be known and bounded.

3.4 Limitations

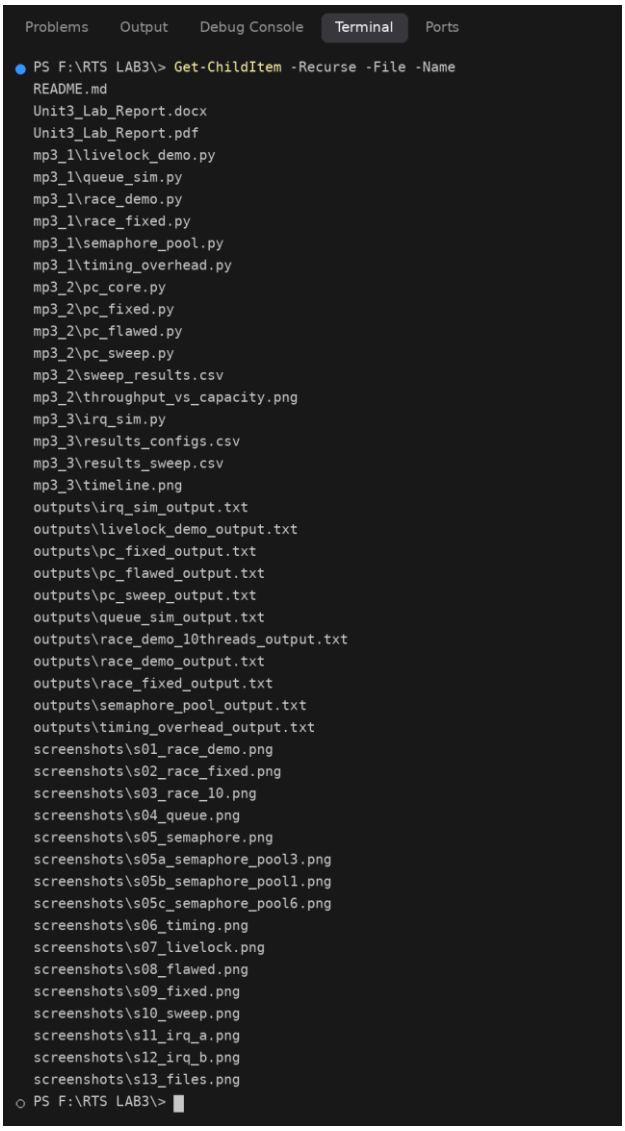
Wall-clock sleeps on a general-purpose OS cannot model microsecond-scale ISR timing, so the model scales everything to milliseconds; sleep granularity (about 1 ms) and OS jitter add noise, especially to maximum values. Runs were 10 s (6 s for the sweep), so the counts of ETH events (99 and 59) are small and the maxima are single events. The 1 ms slice stands in for the interrupt-masked window; on real hardware the equivalent is the longest critical section with interrupts disabled. Despite these limits, the qualitative results, ordering by priority, the damage of long ISRs and growth with arrival rate, were consistent across configurations.

Conclusion

The three projects reproduced the main concurrency hazards of Unit 3 in a controlled setting. Unprotected shared state lost 80% to 90% of updates and was repaired with one lock (3.1). A deadlock with all four Coffman conditions was provoked and then eliminated by a single global lock-ordering rule, and buffer capacity was shown to help only until the critical section became the bottleneck (3.2). A priority-interrupt model showed that long ISRs and high arrival rates at the top of the priority order degrade every task beneath them, with the lowest-priority source's mean latency doubling or worse (3.3). Across all three, worst-case behaviour rather than average behaviour was what mattered.

Appendix A: Source Code

The same files are in the accompanying zip folder, which also contains the raw console outputs, CSV results and charts.



Screenshot 15: Contents of the submission folder.

mp3_1/race_demo.py

```
"""Mini Project 3.1 - Step 1/2: Unsafe shared counter (lost-update race condition, Section 3.8)."""
import threading
import time

shared_counter = 0


def increment_unsafe(iterations):
    global shared_counter
    for _ in range(iterations):
        value = shared_counter          # read
        time.sleep(0.0001)              # widen the interleaving window
        shared_counter = value + 1      # write back (non-atomic read-increment-write)
```

```
def run(num_threads=5, iterations=1000):
    global shared_counter
    shared_counter = 0
    threads = [threading.Thread(target=increment_unsafe, args=(iterations,)) for _ in
range(num_threads)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    return shared_counter



if __name__ == "__main__":
    import sys
    n = int(sys.argv[1]) if len(sys.argv) > 1 else 5
    expected = n * 1000
    result = run(n)
    print(f"threads={n} expected={expected} actual={result} lost_updates={expected - result}")
```

mp3_1/race_fixed.py

```
"""Mini Project 3.1 - Step 4: Same counter protected by a mutex-like Lock (Section 3.4 / 3.8)."""
import threading
import time


shared_counter = 0
counter_lock = threading.Lock()



def increment_safe(iterations):
    global shared_counter
    for _ in range(iterations):
        with counter_lock:              # critical section: read-modify-write is now serialized
            value = shared_counter
            time.sleep(0.0001)
            shared_counter = value + 1



def run(num_threads=5, iterations=1000):
    global shared_counter
    shared_counter = 0
    threads = [threading.Thread(target=increment_safe, args=(iterations,)) for _ in range(num_threads)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    return shared_counter



if __name__ == "__main__":
    # 5 threads x 400 iterations here (the lock serializes the sleep, so 5x1000 would take ~10 s per
run).
    # Pass iterations as argv[1] to use the full 1000.
    import sys
    iters = int(sys.argv[1]) if len(sys.argv) > 1 else 1000
    expected = 5 * iters
    result = run(5, iters)
    print(f"threads=5 iterations={iters} expected={expected} actual={result} lost_updates={expected -
result}")
```

mp3_1/queue_sim.py

```
"""Mini Project 3.1 - Steps 5/6: bounded queue.Queue as an RTOS message queue (Section 3.4)."""
import queue
import statistics
import sys
import threading
import time
```

```
N_ITEMS = 20


def run(maxsize=5, consumer_delay=0.0, producer_delay=0.0):
    q = queue.Queue(maxsize=maxsize)
    latencies = []
    blocked_puts = 0

    def producer_task():
        nonlocal blocked_puts
        for i in range(N_ITEMS):
            if q.full():
                blocked_puts += 1            # producer is about to block (queue full)
            q.put((i, time.perf_counter()))  # blocks automatically when full
            if producer_delay:
                time.sleep(producer_delay)

    def consumer_task():
        for _ in range(N_ITEMS):
            _, sent = q.get()                # blocks when empty
            latencies.append((time.perf_counter() - sent) * 1000.0)  # ms
            if consumer_delay:
                time.sleep(consumer_delay)

    p = threading.Thread(target=producer_task)
    c = threading.Thread(target=consumer_task)
    p.start(); c.start(); p.join(); c.join()
    return min(latencies), max(latencies), statistics.mean(latencies), blocked_puts


def show(label, res):
    mn, mx, avg, blk = res
    print(f"{label:<44} min={mn:8.3f} ms  max={mx:8.3f} ms  avg={avg:8.3f} ms  producer_blocks={blk}")


if __name__ == "__main__":
    print("== Baseline vs slowed consumer (maxsize=5) ==")
    show("fast consumer (no delay)", run(5, 0.0))
    show("slow consumer (5 ms delay)", run(5, 0.005))
    show("slower consumer (20 ms delay)", run(5, 0.020))
    print()
    print("== Lab task: vary maxsize (consumer delay 5 ms) ==")
    print("maxsize | avg latency (ms) | max latency (ms) | producer blocks")
    for size in (1, 2, 5, 10, 20):
        mn, mx, avg, blk = run(size, 0.005)
        print(f"{size:7d} | {avg:16.3f} | {mx:16.3f} | {blk:15d}")
```

mp3_1/semaphore_pool.py

```
"""Mini Project 3.1 - Steps 7/8: counting semaphore as a pool of fixed-size resource blocks (Sections
3.4, 3.6)."""
import random
import statistics
import sys
import threading
import time


def run(pool_size=3, n_tasks=8, seed=1):
    rng = random.Random(seed)
    holds = [rng.uniform(0.05, 0.15) for _ in range(n_tasks)]
    sem = threading.Semaphore(pool_size)
    state_lock = threading.Lock()
    current = 0
    peak = 0
    waits = [0.0] * n_tasks

    def task(i):
```

```
        nonlocal current, peak
        t0 = time.perf_counter()
        sem.acquire()
        waits[i] = (time.perf_counter() - t0) * 1000.0
        with state_lock:
            current += 1
            peak = max(peak, current)
        time.sleep(holds[i])
        with state_lock:
            current -= 1
        sem.release()

    ts = [threading.Thread(target=task, args=(i,)) for i in range(n_tasks)]
    for t in ts:
        t.start()
    for t in ts:
        t.join()
    return waits, peak



if __name__ == "__main__":
    for size in (3, 1, 6):
        waits, peak = run(size)
        immediate = sum(1 for w in waits if w < 1.0)
        waiting = len(waits) - immediate
        print(f"\n-- pool_size={size} --")
        for i, w in enumerate(waits):
            print(f"  task {i}: wait = {w:8.2f} ms")
        print(f"  peak simultaneous holders = {peak} (limit {size})")
        print(f"  tasks proceeding immediately (<1 ms wait) = {immediate}, tasks that waited =
{waiting}")
        print(f"  mean wait = {statistics.mean(waits):.2f} ms, max wait = {max(waits):.2f} ms")
```

mp3_1/timing_overhead.py

```
"""Mini Project 3.1 - Step 9: thread dispatch latency as an analogy to context-switch / ISR latency
(Sections 3.2, 3.5)."""
import statistics
import threading
import time

ITERATIONS = 200



def measure():
    samples = []
    for _ in range(ITERATIONS):
        ev = threading.Event()
        t = threading.Thread(target=ev.set)
        t0 = time.perf_counter()
        t.start()
        ev.wait()
        samples.append((time.perf_counter() - t0) * 1e6)   # microseconds
        t.join()
    return samples



if __name__ == "__main__":
    s = sorted(measure())
    p95 = s[int(0.95 * len(s)) - 1]
    print(f"samples                      : {len(s)}")
    print(f"minimum                      : {s[0]:9.1f} us")
    print(f"mean                         : {statistics.mean(s):9.1f} us")
    print(f"95th percentile              : {p95:9.1f} us")
    print(f"maximum (worst case observed): {s[-1]:9.1f} us")
    print(f"max / mean ratio             : {s[-1] / statistics.mean(s):9.1f}x")
```

mp3_1/livelock_demo.py

```
"""Mini Project 3.1 - Lab Task 3: livelock (Section 3.8).
```

```
Two threads each need BOTH locks but take them in opposite orders. When the second lock is busy they
politely release the first and retry -- but they retry in lockstep (synchronised by barriers), so they
collide every single round. Both stay active (retry counters climb) yet neither completes its work.
A randomised back-off (use_backoff=True) breaks the symmetry and lets them finish.
"""
import random
import threading
import time

ROUNDS = 500



def run(use_backoff=False, rounds=ROUNDS, seed=0):
    lock_a, lock_b = threading.Lock(), threading.Lock()
    retries = {"T1": 0, "T2": 0}
    done = {"T1": False, "T2": False}
    b1, b2, b3 = threading.Barrier(2), threading.Barrier(2), threading.Barrier(2)
    rng = random.Random(seed)

    def worker(name, first, second):
        for _ in range(rounds):
            if done[name]:
                return
            if not use_backoff:
                b1.wait()                               # lockstep start of every attempt
            got_first = first.acquire(blocking=False)
            time.sleep(0.001)                           # both threads now hold their first lock
            if not use_backoff:
                b2.wait()
            got_second = got_first and second.acquire(blocking=False)
            if not use_backoff:
                b3.wait()                               # both attempts finish before either releases
            if got_second:
                done[name] = True                       # real work would happen here
                second.release(); first.release()
                return
            if got_first:
                first.release()                         # polite back-off ...
            retries[name] += 1                          # ... and retry
            if use_backoff:
                time.sleep(rng.uniform(0, 0.005))       # randomised delay breaks lockstep

    t1 = threading.Thread(target=worker, args=("T1", lock_a, lock_b))
    t2 = threading.Thread(target=worker, args=("T2", lock_b, lock_a))
    t0 = time.perf_counter()
    t1.start(); t2.start(); t1.join(); t2.join()
    return retries, done, time.perf_counter() - t0



if __name__ == "__main__":
    r, d, el = run(use_backoff=False)
    print(f"[lockstep retry] after {ROUNDS} rounds ({el:.2f} s): retries T1={r['T1']} T2={r['T2']}; "
          f"work completed T1={d['T1']} T2={d['T2']}")
    print("  -> both threads kept running (retry counters climbing) with zero forward progress:
LIVELOCK")
    r, d, el = run(use_backoff=True)
    print(f"[randomised back-off] finished in {el:.3f} s: retries T1={r['T1']} T2={r['T2']}; "
          f"work completed T1={d['T1']} T2={d['T2']}")
    print("  -> random back-off breaks the lockstep symmetry and both tasks complete")
```

mp3_2/pc_core.py

```
"""Mini Project 3.2 - core bounded-buffer producer/consumer simulation (Section 3.8).

Two explicit locks (no queue.Queue, so acquisition order is fully controlled):
    buffer_lock : guards the bounded buffer (a plain list with a maximum length)
    stats_lock  : guards the shared statistics dictionary

lock_order = "flawed"  : producers take buffer_lock -> stats_lock, consumers take stats_lock ->
buffer_lock
                         (opposite orders => circular wait is possible => deadlock)
```

```
lock_order = "ordered" : EVERY task takes buffer_lock -> stats_lock (global lock ordering => no cycle)

Coffman's four conditions in this code (see report):
  1 mutual exclusion  : threading.Lock is non-shareable
  2 hold-and-wait     : a task holds its first lock while blocking on the second
  3 no preemption     : Python locks cannot be forcibly taken from the holder
  4 circular wait     : only in "flawed" mode (producer holds B waits S, consumer holds S waits B)
"""
import random
import statistics
import threading
import time



class Config:
    def __init__(self, capacity=5, producers=3, consumers=3, total_items=600,
                 lock_order="ordered", hold_gap=0.0005, produce_time=0.001,
                 consume_time=0.001, watchdog_timeout=1.5, seed=42):
        self.capacity = capacity
        self.producers = producers
        self.consumers = consumers
        self.total_items = total_items          # must be divisible by producers and consumers
        self.lock_order = lock_order
        self.hold_gap = hold_gap                # time spent holding the first lock before asking for
the second
        self.produce_time = produce_time        # work done outside any lock
        self.consume_time = consume_time
        self.watchdog_timeout = watchdog_timeout
        self.seed = seed



def run_simulation(cfg, verbose=True):
    buffer = []
    buffer_lock = threading.Lock()
    stats_lock = threading.Lock()
    stats = {"produced": 0, "consumed": 0, "total_wait": 0.0}

    # Bookkeeping used only to build the "lock-wait snapshot" when the watchdog fires.
    state_lock = threading.Lock()
    status = {}                                 # thread name -> (holding, waiting_for)
    finished = threading.Event()
    deadlock = {"detected": False, "snapshot": None}
    prod_waits, cons_waits = [], []

    def set_status(name, holding, waiting):
        with state_lock:
            status[name] = (holding, waiting)

    def acquire(lock, lock_name, name, holding):
        set_status(name, holding, lock_name)    # announce what we are blocked on
        lock.acquire()
        set_status(name, holding + [lock_name], None)

    def release_all(name):
        set_status(name, [], None)

    def producer(pid):
        name = f"producer-{pid}"
        rng = random.Random(cfg.seed + pid)
        for _ in range(cfg.total_items // cfg.producers):
            time.sleep(cfg.produce_time * rng.uniform(0.5, 1.5))      # make an item (no locks held)
            t_start = time.perf_counter()
            while True:
                acquire(buffer_lock, "buffer_lock", name, [])
                time.sleep(cfg.hold_gap)                              # hold-and-wait window
                acquire(stats_lock, "stats_lock", name, ["buffer_lock"])
                if len(buffer) < cfg.capacity:
                    buffer.append(time.perf_counter())
                    stats["produced"] += 1
                    waited = time.perf_counter() - t_start
                    stats["total_wait"] += waited
```

```
                    prod_waits.append(waited)
                    ok = True
                else:
                    ok = False                                      # buffer full: back off and retry
                stats_lock.release()
                buffer_lock.release()
                release_all(name)
                if ok:
                    break
                time.sleep(0.0005)

    def consumer(cid):
        name = f"consumer-{cid}"
        rng = random.Random(cfg.seed + 100 + cid)
        for _ in range(cfg.total_items // cfg.consumers):
            t_start = time.perf_counter()
            while True:
                if cfg.lock_order == "flawed":
                    first, first_n, second, second_n = stats_lock, "stats_lock", buffer_lock,
"buffer_lock"
                else:
                    first, first_n, second, second_n = buffer_lock, "buffer_lock", stats_lock,
"stats_lock"
                acquire(first, first_n, name, [])
                time.sleep(cfg.hold_gap)
                acquire(second, second_n, name, [first_n])
                if buffer:
                    buffer.pop(0)
                    stats["consumed"] += 1
                    waited = time.perf_counter() - t_start
                    stats["total_wait"] += waited
                    cons_waits.append(waited)
                    ok = True
                else:
                    ok = False                                      # buffer empty: back off and
retry
                second.release()
                first.release()
                release_all(name)
                if ok:
                    break
                time.sleep(0.0005)
            time.sleep(cfg.consume_time * rng.uniform(0.5, 1.5))      # use the item (no locks held)

    def watchdog():
        last = (-1, -1)
        last_change = time.perf_counter()
        while not finished.is_set():
            time.sleep(0.1)
            cur = (stats["produced"], stats["consumed"])
            now = time.perf_counter()
            if cur != last:
                last, last_change = cur, now
            elif now - last_change >= cfg.watchdog_timeout and cur[1] < cfg.total_items:
                with state_lock:
                    snap = dict(status)
                deadlock["detected"] = True
                deadlock["snapshot"] = (cur, now - last_change, snap)
                finished.set()
                return

    producers = [threading.Thread(target=producer, args=(i,), daemon=True) for i in
range(cfg.producers)]
    consumers = [threading.Thread(target=consumer, args=(i,), daemon=True) for i in
range(cfg.consumers)]
    wd = threading.Thread(target=watchdog, daemon=True)

    t0 = time.perf_counter()
    for t in producers + consumers + [wd]:
        t.start()

    # Wait for completion or for the watchdog to declare deadlock.
    while not finished.is_set():
```

```
        if all(not t.is_alive() for t in producers + consumers):
            break
        time.sleep(0.05)
    elapsed = time.perf_counter() - t0
    finished.set()

    if deadlock["detected"]:
        (produced, consumed), stalled, snap = deadlock["snapshot"]
        if verbose:
            print("*** WATCHDOG: DEADLOCK DETECTED ***")
            print(f"    no progress for {stalled:.2f} s (produced={produced}, consumed={consumed}, "
                  f"target={cfg.total_items})")
            print("    lock-wait snapshot (thread: holding -> waiting for):")
            for name in sorted(snap):
                holding, waiting = snap[name]
                print(f"      {name:<11}: holding {holding if holding else '[]'} -> waiting for
{waiting}")
        return {"deadlock": True, "snapshot": snap, "elapsed": elapsed,
                "produced": stats["produced"], "consumed": stats["consumed"]}

    throughput = stats["consumed"] / elapsed
    result = {
        "deadlock": False, "elapsed": elapsed, "throughput": throughput,
        "produced": stats["produced"], "consumed": stats["consumed"],
        "avg_prod_block_ms": statistics.mean(prod_waits) * 1000,
        "avg_cons_block_ms": statistics.mean(cons_waits) * 1000,
        "avg_block_ms": statistics.mean(prod_waits + cons_waits) * 1000,
    }
    return result
```

mp3_2/pc_flawed.py

```
"""Flawed version: producers lock buffer->stats, consumers lock stats->buffer. Expected: watchdog
reports deadlock."""
import argparse
from pc_core import Config, run_simulation

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--capacity", type=int, default=5)
    ap.add_argument("--producers", type=int, default=3)
    ap.add_argument("--consumers", type=int, default=3)
    ap.add_argument("--items", type=int, default=600)
    a = ap.parse_args()
    cfg = Config(capacity=a.capacity, producers=a.producers, consumers=a.consumers,
                 total_items=a.items, lock_order="flawed")
    print(f"FLAWED lock order: capacity={a.capacity}, {a.producers} producers, {a.consumers} consumers,
{a.items} items")
    r = run_simulation(cfg)
    print("Result:", "DEADLOCK (simulation stalled, watchdog exited cleanly)" if r["deadlock"] else r)
```

mp3_2/pc_fixed.py

```
"""Corrected version: EVERY task acquires buffer_lock before stats_lock (global lock ordering).
Expected: completes."""
import argparse
from pc_core import Config, run_simulation

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--capacity", type=int, default=5)
    ap.add_argument("--producers", type=int, default=3)
    ap.add_argument("--consumers", type=int, default=3)
    ap.add_argument("--items", type=int, default=600)
    a = ap.parse_args()
    cfg = Config(capacity=a.capacity, producers=a.producers, consumers=a.consumers,
                 total_items=a.items, lock_order="ordered")
    print(f"ORDERED locks (buffer_lock -> stats_lock everywhere): capacity={a.capacity}, "
          f"{a.producers} producers, {a.consumers} consumers, {a.items} items")
    r = run_simulation(cfg)
    if r["deadlock"]:
        print("UNEXPECTED: watchdog fired", r)
    else:
```

```
        print(f"Completed: produced={r['produced']} consumed={r['consumed']} elapsed={r['elapsed']:.2f}
s "
              f"throughput={r['throughput']:.1f} items/s avg_block={r['avg_block_ms']:.2f} ms")
```

mp3_2/pc_sweep.py

```
"""Throughput / blocking-time vs. buffer capacity for the corrected (lock-ordered) simulation. Writes
sweep_results.csv and a chart."""
import csv
import statistics
from pc_core import Config, run_simulation


CAPACITIES = [1, 2, 5, 10, 25, 50]
REPEATS = 3


if __name__ == "__main__":
    rows = []
    print("capacity | throughput (items/s) | avg blocking (ms) | producer block (ms) | consumer block
(ms)")
    for cap in CAPACITIES:
        res = [run_simulation(Config(capacity=cap, lock_order="ordered"), verbose=False) for _ in
range(REPEATS)]
        assert not any(r["deadlock"] for r in res)
        row = {
            "capacity": cap,
            "throughput": statistics.mean(r["throughput"] for r in res),
            "avg_block_ms": statistics.mean(r["avg_block_ms"] for r in res),
            "prod_block_ms": statistics.mean(r["avg_prod_block_ms"] for r in res),
            "cons_block_ms": statistics.mean(r["avg_cons_block_ms"] for r in res),
        }
        rows.append(row)
        print(f"{cap:8d} | {row['throughput']:20.1f} | {row['avg_block_ms']:17.2f} | "
              f"{row['prod_block_ms']:19.2f} | {row['cons_block_ms']:19.2f}")
    with open("sweep_results.csv", "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
        w.writeheader(); w.writerows(rows)
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        fig, ax1 = plt.subplots(figsize=(7, 4))
        caps = [r["capacity"] for r in rows]
        ax1.plot(caps, [r["throughput"] for r in rows], "o-", color="#1f4e79", label="Throughput")
        ax1.set_xscale("log"); ax1.set_xlabel("Buffer capacity (items, log scale)")
        ax1.set_ylabel("Throughput (items/s)", color="#1f4e79")
        ax2 = ax1.twinx()
        ax2.plot(caps, [r["avg_block_ms"] for r in rows], "s--", color="#c55a11", label="Avg blocking
time")
        ax2.set_ylabel("Avg blocking time (ms)", color="#c55a11")
        ax1.set_title("Throughput and blocking time vs buffer capacity (3 producers, 3 consumers)")
        fig.tight_layout(); fig.savefig("throughput_vs_capacity.png", dpi=150)
        print("chart saved: throughput_vs_capacity.png")
    except ImportError:
        print("matplotlib not installed - use sweep_results.csv to chart in a spreadsheet")
```

mp3_3/irq_sim.py

```
"""Mini Project 3.3 - Priority-based interrupt latency simulator (Section 3.5).


Model (all times in milliseconds, measured with time.perf_counter):
  * Each interrupt source is a periodic thread. At each release it records the ASSERTION timestamp.
  * TOP HALF  : runs as soon as the simulated CPU can accept an interrupt, i.e. when no other top half
is
                running and the current bottom-half *slice* (the "interrupt-masked window", SLICE_MS)
ends.
                Top halves ALWAYS preempt bottom-half work. Interrupt latency = top-half start -
assertion.
  * BOTTOM HALF: at the end of the top half a bottom-half task (priority, assertion time, duration) is
pushed
                onto a heapq priority queue. ONE CPU worker always runs the highest-priority ready
bottom half
                in SLICE_MS slices; between slices it yields to any pending top half and, if preempted
by a
                higher-priority bottom half, the remainder is re-queued.
```

```
               Response latency = bottom-half completion - assertion (Section 3.5).
Simplification: top halves are serialised (not nested); a top half waits for the running one to finish.
"""
import csv
import heapq
import statistics
import threading
import time
from dataclasses import dataclass

SLICE_MS = 1.0     # granularity at which the CPU can switch (models the interrupt-masked window)



@dataclass
class Source:
    name: str
    priority: int        # lower number = higher priority
    period_ms: float
    top_ms: float
    bottom_ms: float



def now_ms():
    return time.perf_counter() * 1000.0



def sleep_ms(ms):
    if ms > 0:
        time.sleep(ms / 1000.0)



def simulate(sources, duration_s=10.0, record_timeline=False):
    cv = threading.Condition()
    heap = []                                   # (priority, assertion_ms, seq, source_name,
remaining_ms)
    state = {"isr_running": False, "isr_waiting": 0, "in_slice": False, "stop": False, "seq": 0}
    log = {s.name: {"irq_lat": [], "top": [], "resp": []} for s in sources}
    timeline = []
    t_origin = now_ms()

    def source_thread(src):
        next_release = now_ms() + src.period_ms * (0.3 + 0.2 * src.priority)   # staggered start
        while True:
            sleep_ms(next_release - now_ms())
            if state["stop"]:
                return
            asserted = now_ms()
            with cv:
                state["isr_waiting"] += 1
                while state["isr_running"] or state["in_slice"]:
                    cv.wait()
                state["isr_waiting"] -= 1
                state["isr_running"] = True
            top_start = now_ms()
            sleep_ms(src.top_ms)                # top-half work (acknowledge, capture data)
            top_end = now_ms()
            with cv:
                state["isr_running"] = False
                state["seq"] += 1
                heapq.heappush(heap, (src.priority, asserted, state["seq"], src.name, src.bottom_ms))
                cv.notify_all()
            log[src.name]["irq_lat"].append(top_start - asserted)
            log[src.name]["top"].append(top_end - top_start)
            if record_timeline:
                timeline.append((top_start - t_origin, top_end - t_origin, src.name, "top"))
            next_release += src.period_ms

    def cpu_worker():
        while True:
            with cv:
```

```
                while not state["stop"] and (not heap or state["isr_running"] or state["isr_waiting"] >
0):
                    cv.wait(0.05)
                if state["stop"]:
                    return
                prio, asserted, seq, name, remaining = heapq.heappop(heap)   # highest-priority ready
bottom half
                state["in_slice"] = True
            run = min(SLICE_MS, remaining)
            s0 = now_ms()
            sleep_ms(run)
            s1 = now_ms()
            with cv:
                state["in_slice"] = False
                remaining -= run
                if remaining > 1e-9:
                    heapq.heappush(heap, (prio, asserted, seq, name, remaining))   # preempted /
unfinished
                else:
                    log[name]["resp"].append(now_ms() - asserted)
                cv.notify_all()
            if record_timeline:
                timeline.append((s0 - t_origin, s1 - t_origin, name, "bottom"))

    threads = [threading.Thread(target=source_thread, args=(s,), daemon=True) for s in sources]
    worker = threading.Thread(target=cpu_worker, daemon=True)
    worker.start()
    for t in threads:
        t.start()
    time.sleep(duration_s)
    with cv:
        state["stop"] = True
        cv.notify_all()
    time.sleep(0.2)
    return log, timeline



def pct(data, p):
    s = sorted(data)
    return s[min(len(s) - 1, max(0, int(round(p / 100 * len(s))) - 1))]



def summarize(label, sources, log):
    rows = []
    print(f"\n=== {label} ===")
    print(f"{'source':<8}{'prio':>5}{'n':>6} | {'resp min':>9}{'mean':>9}{'p95':>9}{'max':>9} | {'irq-
lat mean':>13}{'max':>8} | {'top mean':>9}")
    for s in sources:
        d = log[s.name]
        if not d["resp"]:
            continue
        r, il, tp = d["resp"], d["irq_lat"], d["top"]
        row = dict(config=label, source=s.name, priority=s.priority, n=len(r),
                   resp_min=min(r), resp_mean=statistics.mean(r), resp_p95=pct(r, 95), resp_max=max(r),
                   irq_mean=statistics.mean(il), irq_max=max(il), top_mean=statistics.mean(tp))
        rows.append(row)
        print(f"{s.name:<8}{s.priority:>5}{len(r):>6} |
{row['resp_min']:9.2f}{row['resp_mean']:9.2f}{row['resp_p95']:9.2f}"
              f"{row['resp_max']:9.2f} | {row['irq_mean']:13.2f}{row['irq_max']:8.2f} |
{row['top_mean']:9.2f}   (ms)")
    return rows



def base_sources(s1_period=20.0, s2_top=0.5, s1_top=0.5, s3_top=1.0):
    return [
        Source("UART",  1, s1_period, s1_top, 3.0),     # highest priority
        Source("ADC",   2, 50.0,      s2_top, 8.0),
        Source("ETH",   3, 100.0,     s3_top, 15.0),    # lowest priority
    ]
```

```
if __name__ == "__main__":
    import sys
    DUR = float(sys.argv[1]) if len(sys.argv) > 1 else 10.0
    SWEEP_DUR = float(sys.argv[2]) if len(sys.argv) > 2 else 6.0
    all_rows = []

    # Config A: well-behaved baseline (short top halves)
    src = base_sources()
    logA, timeline = simulate(src, DUR, record_timeline=True)
    all_rows += summarize("A: baseline (short top halves)", src, logA)

    # Config B: ADC (middle priority) has a deliberately long top half (violates Section 3.5 guidance)
    src = base_sources(s2_top=12.0)
    logB, _ = simulate(src, DUR)
    all_rows += summarize("B: long ADC top half (12 ms)", src, logB)

    # Config C: long top half on the highest-priority source
    src = base_sources(s1_top=8.0)
    logC, _ = simulate(src, DUR)
    all_rows += summarize("C: long UART top half (8 ms)", src, logC)

    # Arrival-rate sweep: shorten the highest-priority source's period, watch the lowest-priority
source (ETH)
    print("\n=== Sweep: UART period vs ETH (lowest priority) response latency ===")
    print(f"{'UART period (ms)':>17}{'UART load %':>13}{'ETH n':>7}{'ETH mean':>10}{'ETH p95':>9}{'ETH
max':>9}")
    sweep_rows = []
    for period in (40.0, 20.0, 12.0, 8.0):
        src = base_sources(s1_period=period)
        lg, _ = simulate(src, SWEEP_DUR)
        r = lg["ETH"]["resp"]
        load = (3.0 + 0.5) / period * 100
        sweep_rows.append(dict(uart_period_ms=period, uart_load_pct=load, eth_n=len(r),
                               eth_mean=statistics.mean(r), eth_p95=pct(r, 95), eth_max=max(r)))

print(f"{period:17.1f}{load:13.1f}{len(r):7d}{statistics.mean(r):10.2f}{pct(r,95):9.2f}{max(r):9.2f}")

    with open("results_configs.csv", "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(all_rows[0].keys())); w.writeheader();
w.writerows(all_rows)
    with open("results_sweep.csv", "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(sweep_rows[0].keys())); w.writeheader();
w.writerows(sweep_rows)

    # Optional timeline plot: 200 ms window from the baseline run showing preemption
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        lo = 2000.0
        hi = lo + 200.0
        names = ["UART", "ADC", "ETH"]
        colors = {"UART": "#1f4e79", "ADC": "#c55a11", "ETH": "#548235"}
        fig, ax = plt.subplots(figsize=(10, 3.6))
        for (a, b, n, kind) in timeline:
            if b < lo or a > hi:
                continue
            y = names.index(n)
            ax.barh(y + (0.2 if kind == "top" else -0.2), b - a, left=a, height=0.35,
                    color=colors[n], alpha=1.0 if kind == "top" else 0.55,
                    hatch="//" if kind == "top" else None, edgecolor="black", linewidth=0.3)
        ax.set_xlim(lo, hi); ax.set_yticks(range(3)); ax.set_yticklabels(names)
        ax.set_xlabel("time since start (ms)")
        ax.set_title("Baseline timeline: hatched = top half (ISR), solid = bottom-half slices (1 ms
each)")
        fig.tight_layout(); fig.savefig("timeline.png", dpi=150)
        print("\ntimeline saved: timeline.png")
    except ImportError:
        print("matplotlib not installed; skipping timeline plot")
```